Обзор решения keepalived + postgresql_ha_controller.sh

Введение

Рассматриваемое решение предназначено для поддержания работоспособности репликационной пары серверов PostgreSQL, сконфигурированной в режиме потоковой репликации.

Работа решения возможна в двух режимах:

- sync-only – нацелен на повышенную сохранность данных, путем поддержания репликационной пары в состоянии синхронной репликации.
- standalone-master – нацелен на ускоренное восстановление при отказе. В данном режиме разрешена асинхронная репликация и самостоятельная работа мастер-сервера.

Сценарии обработки отказов см. стр.12

Архитектура решения представляет собой 2 идентичных скрипта, функционирующих на обоих хостах кластера, и демона keepalived, обеспечивающего доступ по VIP к текущему активному мастеру. Скрипты независимо запускаются утилитой **cron** на каждом из хостов, определяют состояние репликационной пары postgres, и, при обнаружении нарушений, совместно работают над их устранением. Согласованность действий между инстансами скрипта локально и на разных хостах обеспечивается при помощи механизма сигналов и блокировок.

Установка

Подготовка PostgreSQL

Для корректной работы скрипта необходимо обеспечить:

- Одинаковое расположение и наименование DATA директорий PostgreSQL на двух хостах.
- Беспарольный доступ к суперпользователю СУБД - postgres и пользователю, обладающему правами на репликацию. Беспарольный доступ необходимо обеспечить для системных пользователей **root** и **postgres**.
  Для этого можно поместить в домашние директории системных пользователей файл “.pgpass” с правами 600, в котором будут записаны пароли к пользователям СУБД в формате “hostname:port:database:username:password”:
  ```
  # hostname:port:database:username:password
  *:*:*:postgres:postgres
  *:*:*:replicator:replicator
  ```
  Также необходимо разрешить подключение к СУБД с каждого из хостов для суперпользователя и пользователя, обладающего правами на репликацию, добавив соответствующие записи в pg_hba.conf.
- Конфигурация инстансов PostgreSQL должна в полной мере содержаться в **postgresql.conf**, так как скрипт использует **postgresql.auto.conf** и **recovery.conf** в процессе работы для настройки репликации (содержимое файлов при работе скрипта очищается).

Подготовка системы

Подготовьте рабочую директорию для скрипта:

В ней необходимо разместить директории для системы блокировок и для сохранения архива логов PostgreSQL, а также сами скрипты:

- **postgresql_ha_controller.sh**
- **pg_ha_startup_cleanup.sh** (внутри скрипта необходимо задать актуальный путь к директории системы блокировок)
- **start_script.sh** (при необходимости, указать актуальный путь к скрипту postgresql_ha_controller.sh)

!!! Все директории и файлы скрипта необходимо создавать с правами 700 root:root.

Примерный вид рабочей директории:

```
drwx------   2 root root   4096 Apr  7 07:46 locks
-rwx------   1 root root    394 Mar 28 15:49 pg_ha_startup_cleanup.sh
drwx------   2 root root   4096 Apr  7 07:30 pg_log_archive
-rwx------   1 root root 114262 Mar 28 16:36 postgresql_ha_controller.sh
-rwx------   1 root root    390 Mar 28 15:49 start_script.sh
```

Для корректной работы скрипта после перезагрузки системы, необходимо добавить сервис **pg_ha_startup_cleanup.service**, например в /etc/systemd/system/.

В файле **pg_ha_startup_cleanup.service** необходимо указать актуальный путь к скрипту **pg_ha_startup_cleanup.sh.**

После чего перечитать конфигурации сервисов: `systemctl daemon-reload`

И установить новый сервис в автозапуск: `systemctl enable --now pg_ha_startup_cleanup.service`

Требования к конфигурации системы:

Необходимо обеспечить двухсторонний беспарольный доступ для системного администратора **root** между двумя хостами.

Для корректной работы скрипта, требуется версия bash 4.0 или выше.

Также на системе должны быть доступны следующие командные утилиты:

- awk
- basename
- cat
- chmod
- chown
- date

- echo
- find
- grep
- head
- read
- rm
- sleep
- ssh
- sudo (должен быть беспарольным для root)
- timeout

Подготовка keepalived

Для обеспечения доступа к текущему мастер-серверу PostgreSQL, необходимо установить и настроить keepalived на обоих хостах.

После установки поместите в /etc/keepalived/ скрипт мониторинга статуса репликации PostgreSQL – **postgresql_replication_role_check.sh** с правами 700 root:root. При помощи этого скрипта keepalived будет выбирать сервер с мастером PostgreSQL, на который необходимо поместить VIP. Внутри скрипта необходимо указать актуальные абсолютные пути к утилитам **pg_isready** и **psql**.

Для настройки сервиса keepalived создайте файл /etc/keepalived/keepalived.conf

Пример конфигурации на мастере:

```
global_defs {
        script_user root
        enable_script_security
}

vrrp_script check_pg_is_master {
        script "/etc/keepalived/pg_replication_role_check.sh"
        interval 3
        timeout 2
        fall 1
        rise 1
}

vrrp_instance PG_VIP {
        state MASTER
        interface eth1
        virtual_router_id 97
        priority 255
        advert_int 1
        authentication {
                auth_type PASS
                auth_pass p@$$word
        }
        virtual_ipaddress {
                192.168.56.100/24
        }
        track_script {
                check_pg_is_master
        }
}
```

Пример конфигурации на реплике:

```
global_defs {
        script_user root
        enable_script_security
}

vrrp_script check_pg_is_master {
        script "/etc/keepalived/pg_replication_role_check.sh"
        interval 3
        timeout 2
        fall 1
        rise 1
}

vrrp_instance PG_VIP {
        state BACKUP
        interface eth1
        virtual_router_id 97
        priority 254
        advert_int 1
        authentication {
                auth_type PASS
                auth_pass p@$$word
        }
        virtual_ipaddress {
                192.168.56.100/24
        }
        track_script {
                check_pg_is_master
        }
}
```

В конфигурации необходимо указать:

- **script_user** – пользователь, от которого будет запускаться скрипт мониторинга репликационного статуса PostgreSQL. Необходимо указать системного администратора root.
- **script** – путь к скрипту мониторинга репликационного статуса PostgreSQL.
- **interval** – периодичность, с которой keepalived будет проверять состояние PostgreSQL.
- **timeout** – таймаут, после которого инстанс признается неподходящим для VIP.
- **fall** – количество отрицательных поверок, необходимых для признания хоста не валидным для VIP. Правильно ставить значение 1.

- **rise** – количество положительных проверок, необходимых для признания хоста пригодным для VIP. Правильно ставить значение 1.
- **state** – целевое состояние VRRP инстанса. Для исходного сервера-мастера – MASTER, для исходного сервера-реплики – BACKUP.
- **interface** – сетевой интерфейс, к которому привязывается VIP адрес.
- **virtual_router_id** – идентификационный номер VRRP роутера, должен быть уникален внутри сети и одинаков да обоих серверах.
- **priority** – указывает приоритет переноса VIP в случае, если оба сервера пригодны. Должен быть выше на мастере.
- **auth_pass** – пароль для VRRP инстанса. Должен быть одинаковым на обоих серверах.
- **virtual_ipaddress** – ip адрес с маской подсети, который и будет VIP.

Подготовка скрипта

Конфигурационные параметры скрипта представлены в начале файла в блоке SETTINGS. Все параметры обязательны к заполнению.

Files & directories:

- LOGFILE_PATH – путь к файлу логов скрипта (файл будет создан автоматически).
- LOCK_FILE_DIR – путь к директории системы сигналов и блокировок.
- KEEPALIVED_DIR – путь к директории конфигурации keepalived.
- PG_LOG_ARCIVE_DIR – путь к директории для сохранения логов PostgreSQL.

- PGDATA – путь к DATA директории PostgreSQL.
- PGWAL – путь к pg_wal директории PostgreSQL.
- PGLOG – имя директории, хранящей логи, в каталоге PGDATA.

- PG_ISREADY – путь к утилите pg_isready.
- PG_CTL – путь к утилите pg_ctl.
- PSQL – путь к утилите psql.
- PG_CONTROLDATA – путь к утилите pg_controldata.
- PG_BASEBACKUP – путь к утилите pg_basebackup.
- PG_REWIND – путь к утилите pg_rewind.

Users:

- SYSTEM_ADMIN_USER – системный администратор (предполагается root).
- SYSTEM_POSTGRES_USER – системный пользователь-администратор PostgreSQL (предполагается postgres).
- PG_ADMIN_USER – внутренний пользователь-администратор СУБД (предполагается postgres).
- PG_REPLICATION_USER – внутренний пользователь СУБД с правами на репликацию.

Cluster information:

- CLUSTER_NAME_LOCAL – имя локального инстанса PostgreSQL.
- CLUSTER_NAME_REMOTE – имя удаленного инстанса PostgreSQL.
- CLUSTER_IP_REMOTE – ip адрес удаленного инстанса PostgreSQL.
- PGPORT – порт, на котором слушает PostgreSQL (по умолчанию 5432).

Retries & timeouts

Для настроек ниже, время указывается в секундах.

При установке времени таймаута в 0, механизм таймаута отключается.

- LOCK_RETRIES – количество попыток при создании блокировки скрипта или сигнала.
- UNLOCK_RETRIES – количество попыток при удалении блокировки скрипты или сигнала.
- SSH_CONNECT_TIMEOUT – таймаут на время подключения по SSH, после которого SSH интерконнект признается нерабочим.
- PG_STATUS_CHECK_RETRIES – количество попыток проверки текущего состояния сервера PostgreSQL.
- PG_STATUS_CHECK_TIMEOUT – таймаут после которого сервер PostgreSQL признается нерабочим.
- PG_REPLCATION_CHECK_RETRY_DELAY – задержка между проверками статуса репликации, после восстановления.
- PG_REPLICATION_CHECK_TIMEOUT – таймаут для проверки статуса репликации восстановленного кластера PostgreSQL, после которого восстановление признается неуспешным.
- PG_BASEBACKUP_RESTORE_TIMEOUT – таймаут для операции восстановления из бэкапа, после чего операция признается неуспешной и прерывается.
- PG_START_TIMEOUT – таймаут для операции запуска кластера PostgreSQL, после которого запуск признается неуспешным и завершается.
- PG_STOP_TIMEOUT - таймаут для операции остановки кластера PostgreSQL, после которого остановка признается неуспешной и завершается.
- PG_REWIND_TIMEOUT - таймаут для операции отката старого матера до состояния текущего, после которого откат признается неуспешным и завершается.
- STATE_VERIFICATION_TIMEOUT – таймаут для процесса согласования состояния кластера.
- STATE_VERIFICATION_RETRY_DELAY – задержка между попытками согласовать состояние кластера.
- RETRY_DELAY – глобальное время ожидания между попытками.

Script behavior

- LOG_LEVEL – уровень логирования скрипта (рекомендованный уровень логирования 5 - Notice, на нем скрипт будет записывать в логи только информацию об активных действиях, выполняемых во время починки репликационной пары PostgreSQL).
  0 – Emergency
  1 – Alert
  2 – Critical [FATAL]
  3 – Error [ERROR]
  4 – Warning [WARNING]
  5 – Notice
  6 – Informational [INFO]
  7 – Debug [DEBUG]
- LOG_TO_JOURNAL – контролирует функционал логирования в системный журнал. 1 – включает, 0 – выключает.
- JOURNAL_LOG_LEVEL – аналогично опции LOG_LEVEL, только для логирования в системный журнал (рекомендованный уровень логирования – 5 - Notice).
- PG_LOG_ACHIVING – 1 – включает опцию сохранения логов локального инстанса PostgreSQL в архив перед деструктивными действиями над каталогом PGDATA. 0 – выключает архивацию логов.
- PG_ALLOW_STANDALONE_MASTER – контролирует основной функционал скрипта. 0 – переводит скрипт в режим sync-only, 1 – переводит скрипт в режим standalone-master.
- PG_USE_RECOVERY_CONF – изменяет метод активации standby режима. 1 – использует recovery.conf для PostgreSQL 11, 0 – использует standby.signal для версий PostgreSQL 12+.
- PG_CATCHUP_TIMEOUT – время в минутах, которое мастеру разрешено находиться без синхронной реплики (т.е. либо без реплики, либо с догоняющей\отстающей асинхронной репликой) актуально если скрипт работает в режиме standalone-master.
- PG_REPLICATION_SLOT_NAME – имя слота репликации, используемого скриптом.

Запуск и использование

1. После установки и настройки согласно описанным выше требованиям, запустите оба кластера PostgreSQL в режиме синхронной репликации.
2. Запустите сервис keepalived и добавьте его в автозапуск.
   systemctl enable --now keepalived
3. Убедитесь, что сервис cron запущен и добавлен в автозапуск
   systemctl enable --now cron
4. Добавьте в cron запись на запуск скрипта **postgresql_ha_controller.sh** (или о запуске через прокси скрипт **start_script.sh**)

Важно!

Если потребуется приостановить работу скрипта, можно приостановить задачу в cron, однако правильнее будет создать любой файл в директории LOCK_FILE_DIR с расширением “.lock” например “maintenance.lock”. При его удалении работа системы продолжится в штатном режиме.

Обзор сценариев обработки отказа системы

Sync-only mode:

Сценарий 1.1 - Отказ реплики с сохранением интерконнекта:

(СУБД недоступна)

1. Скрипты согласуют состояние кластера
2. Восстановление реплики
   2.1. Производится попытка перезапуска реплики
   2.2. Производится восстановление реплики через pg_basebackup
3. Проверяется наличие синхронной репликации

(СУБД доступна)

Сценарий 1.2 - Отказ мастера с сохранением интерконнекта:

(СУБД недоступна)

1. Скрипты согласуют состояние кластера
2. Реплика повышается до нового мастера
3. Присоединение старого мастера в качестве новой реплики
   3.1. Производится попытка откатить предыдущий мастер pg_rewind
   3.2. Новая реплика восстанавливается через pg_basebackup
4. Проверяется наличие синхронной репликации

(СУБД доступна)

Сценарий 1.3 - Одновременный отказ мастера и реплики с сохранением интерконнекта:

(СУБД недоступна)

1. Скрипты согласуют состояние кластера
2. Перезапуск упавшего мастера
   2.1. Если запустился, переходим к сценарию - 1.1
3. Перезапуск упавшей реплики
   3.1. Если запустилась, переходим к сценарию - 1.2
4. Система не подлежит автоматическому восстановлению

Сценарий 1.4 - Отказ SSH интерконнекта:

1. Фиксируется пропажа SSH соединения
2. Скрипты переходят в режим ожидания до восстановления соединения
3. Соединение восстанавливается, скрипт переходит к стандартной работе.

Standalone-master mode:

<u>Сценарий 2.1 - Отказ реплики с сохранением интерконнекта:</u>

(СУБД недоступна)

1. Скрипты согласуют состояние кластера
2. Мастер переходит в режим самостоятельной работы

(СУБД доступна)

3. Восстановление реплики
   3.1. Производится попытка перезапуска реплики
   3.2. Производится восстановление реплики через pg_basebackup
4. Проверяется наличие асинхронной репликации
5. Запускается процесс добавления реплики как синхронной
   5.1. Реплика догнала мастер и успешно стала синхронной
   5.2. Реплика фатально отстала от мастера, происходит восстановление через pg_basebackup
6. Проверяется синхронная репликация

<u>Сценарий 2.2 - Отказ мастера с сохранением интерконнекта:</u>

(СУБД недоступна)

1. Скрипты согласуют состояние кластера
2. Реплика повышается до нового мастера

(СУБД доступна)

3. Присоединение старого мастера в качестве новой реплики
   3.1. Производится попытка откатить предыдущий мастер pg_rewind
   3.2. Новая реплика восстанавливается через pg_basebackup
4. Проверяется наличие асинхронной репликации
5. Запускается процесс добавления реплики как синхронной
   5.1. Реплика догнала мастер и успешно стала синхронной
   5.2. Реплика фатально отстала от мастера, происходит восстановление через pg_basebackup
6. Проверяется синхронная репликация

<u>Сценарий 2.3 - Одновременный отказ мастера и реплики с сохранением интерконнекта:</u>

(СУБД недоступна)

1. Скрипты согласуют состояние кластера

2. Перезапуск упавшего мастера
   2.1. Если запустился, переходим к сценарию - 2.1
3. Перезапуск упавшей реплики
   3.1. Если запустилась, переходим к сценарию - 2.2
4. Система не подлежит автоматическому восстановлению

<u>Сценарий 2.4 - Отказ SSH интерконнекта:</u>

(СУБД недоступна)

1. Мастер переходит в режим самостоятельной работы
2. Скрипты переходят в режим ожидания до восстановления соединения.
3. Соединение восстанавливается
4. Запускается процесс добавления реплики как синхронной
   4.1. Реплика догнала мастер и успешно стала синхронной
   4.2. Реплика фатально отстала от мастера, происходит восстановление через pg_basebackup
5. Проверяется синхронная репликация